argus

```
cockpit/
├── frontend/          # React app
│   ├── components/
│   │   ├── HomePanel.jsx
│   │   ├── Terminal.jsx      # xterm.js wrapper
│   │   ├── AgentPane.jsx     # right side, both agents
│   │   └── SessionCard.jsx   # investigation cards
│   └── store/         # Zustand state
├── backend/           # FastAPI
│   ├── agents/
│   │   ├── offsec.py
│   │   └── reporter.py
│   ├── sessions/      # code lookup, summary store
│   └── terminal/      # node-pty bridge
└── db/                # SQLite
```

argus — AI-Powered Pentester's Workstation

Overview

argus is a web-based terminal environment designed for offensive security researchers. It combines real CLI access with two specialized AI agents in a split-pane interface — one focused on active security research, the other on tracking and reporting. The goal is to give a solo pentester everything they need in one place: a live terminal, an AI collaborator, and a persistent engagement tracker, without switching between tools.

Interface Layout

The screen is divided into two sections. The left side contains the terminal workspace. The right side houses the two AI agents.

A home/side panel lists all recent investigations as cards. Each investigation is assigned a unique 6-digit code (e.g. `INV-482931`) on creation. The researcher can start a new investigation or resume an existing one directly from this panel.

Terminal Layer

Each investigation gets a primary terminal session tied to its 6-digit code. Within an investigation, the researcher can spawn multiple sub-terminals for parallel workstreams — running different tools, targeting different surfaces simultaneously. Each sub-terminal gets its own sub-code (e.g. `482931-01`, `482931-02`) for identification.

All terminal sessions run real shell access via node-pty — not a simulated chat interface.

The Two Agents

Agent 1 — OffSec Researcher

Active collaborator during engagements. Assists with crafting payloads, interpreting tool output, suggesting attack paths, researching CVEs, and developing exploits. Operates during the "doing" phase of an engagement.

Agent 2 — Reporter & Tracker

Passive until called. Maintains structured state across all investigations: what vulnerabilities have been found, what's been submitted, what's currently ongoing, and what targets are planned next. Generates report drafts on demand. Gives the researcher a fast situational overview when returning after a break.

Session Memory & Agent Context

Each investigation code maps to a stored session object containing a structured summary of everything that has happened — targets, open ports, findings, tool outputs, sub-terminal activity, and report state.

When the researcher switches agents or resumes a session, they provide the relevant code. The backend fetches the session summary for that code and injects it as a system prompt prefix for the agent. The agent responds with full context of what has happened so far without requiring the researcher to re-explain anything.

Summaries are written for the agent to read — structured and inference-ready, not raw logs.

Data Model (per investigation)

```
INV-XXXXXX
├── created_at, scope, status
├── session_summary
├── findings[]
├── sub_terminals[]
│   ├── XXXXXX-01: { purpose, summary, findings[] }
│   └── XXXXXX-02: { purpose, summary, findings[] }
└── report_state { submitted[], ongoing[], planned[] }
```

Stack

Layer	Technology
Frontend	React + xterm.js + Zustand
Backend	Python + FastAPI
Terminal	node-pty over WebSocket
Session Store	SQLite (v1)
AI	Anthropic API (Claude), two system prompts

Why It's Useful

Most pentesters manage their workflow across a terminal, a notes app, and a separate reporting tool — manually stitching context between all three. Argus collapses that into a single interface where the terminal, the AI collaborator, and the engagement tracker share the same underlying session state. The researcher stays in flow, and nothing gets lost between sessions.

ideas for argus

ARGUS — Project Overview & v0.0 Specification

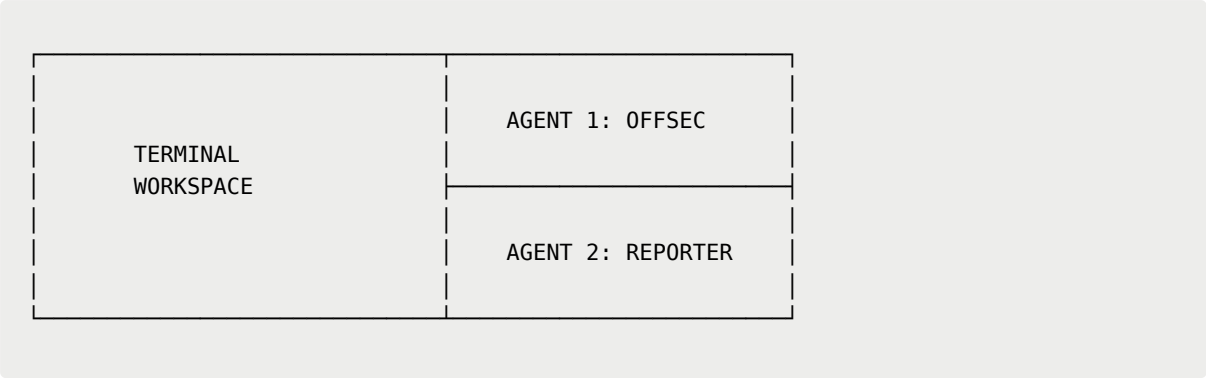
What Argus Is

Argus is a web-based pentester's workstation built for solo offensive security researchers. It combines a real terminal environment, two specialized AI agents, and a persistent engagement tracker into a single interface. The name comes from Argus Panoptes — the hundred-eyed giant of Greek mythology — chosen for its connotation of total visibility over an engagement.

Design philosophy: **Terminal first. AI second.** The researcher is always in control. The agents are tools, not the focus.

The problem it solves: solo pentesters currently stitch together a terminal, a notes app, and a reporting tool manually. Argus collapses all three into one interface where the terminal, the AI collaborator, and the engagement tracker share the same underlying session state.

Interface Layout



Side / Home Panel
Collapsible panel listing all investigations as cards. Each card shows the investigation code, target scope, status, and last activity. Options to create a new investigation or resume an existing one.

Left Pane — Terminal
Real shell access. The researcher runs their tools here. Resizable. This is the primary workspace.

Right Pane — Agent Panel
Two AI agents, one above the other or toggled. Collapsible. The researcher interacts with whichever agent is relevant — OffSec during active work, Reporter when logging findings or reviewing status.

Investigation System

Every investigation is assigned a unique 6-digit code on creation (e.g. `INV-482931`). This code is the primary lookup key for everything — session memory, findings, notes, timeline, and report state.

Investigations have three statuses: **Active**, **Paused**, **Completed**.

Last activity timestamp updates automatically on every action.

The Two Agents

Agent 1 — OffSec Researcher

Active collaborator during engagements. Assists with crafting payloads, interpreting tool output from the terminal, suggesting attack paths, researching CVEs, and planning recon. Operates during the doing phase of an engagement.

Agent 2 — Reporter & Tracker

Passive until called. Maintains structured state across all investigations — what vulnerabilities have been found, what's open, what's been submitted, what's resolved. Generates investigation summaries and Markdown report drafts on demand. Gives the researcher a fast situational overview when returning after a break.

How both agents know what's happening:

When the researcher opens an investigation, the backend fetches the session summary for that investigation code and injects it as a system prompt prefix for whichever agent is active. The agent responds with full context of everything that has happened — targets, findings, notes, timeline — without the researcher needing to re-explain anything.

Summaries are written for the agent to read, not the human. Structured and inference-ready:

```
Target: example.com
Scope: *.example.com, 10.0.0.0/24
Status: Active
Findings: Reflected XSS on /search (medium, open), Admin panel exposed (high, open)
Recent notes: Login endpoint not yet tested
Last activity: ffuf scan on /api — no results
```

Terminal-Native Commands

Researchers commit findings and notes directly from the terminal without switching context. Lines starting with `/commit` are intercepted by the backend before being passed to the shell:

```
/commit finding "Reflected XSS on search endpoint" --severi
ty medium
/commit note "Admin panel exposed on subdomain, investigate
further"
/commit asset "admin.example.com"
```

The Reporter agent converts these into structured records automatically and the timeline updates on every commit.

Findings Structure

Every finding is a structured record:

```
finding
├── id
├── title
├── severity        (critical / high / medium / low / info)
├── status          (open / submitted / resolved)
├── description
├── affected_assets[]
├── reproduction_steps
├── remediation
└── discovered_at
```

Activity Timeline

Every investigation maintains a chronological event log that updates automatically:

```
09:10  Investigation created — example.com
09:15  Asset committed — admin.example.com
09:40  Finding committed — Reflected XSS (medium)
10:05  Note committed — Login endpoint untested
10:30  Report draft generated
```

Investigation Cards (Home Panel)

Each card shows:

```
INV-482931 — example.com
Status       : Active
Findings     : 4
Last Activity: 17 mins ago
```

Reporting

The Reporter agent generates an investigation summary and a structured Markdown report on demand, pulling from the findings and notes database — not from raw conversation. Export is clean and consistent regardless of how messy the session was.

Argus v0.0 — What's Included

Area	Feature	In v0.0
Investigations	Create investigation	✅
Investigations	Resume investigation	✅
Investigations	Status tracking (Active/Paused/Completed)	✅
Investigations	Last activity tracking	✅
Investigations	Home panel with cards	✅
Terminal	Real terminal via node-pty	✅
Terminal	WebSocket streaming	✅
Terminal	Single terminal per investigation	✅
Terminal	Sub-terminals	❌ v0.1
Terminal	Focus mode	❌ v0.1
AI	OffSec agent	✅
AI	Reporter agent	✅
AI	Code-based context injection	✅
AI	Session summaries	✅
AI	Context compression	❌ v0.1
Findings	Structured findings database	✅
Findings	Severity levels	✅
Findings	Finding status	✅
Findings	Affected assets	✅
Findings	Evidence attachments	❌ v0.1
Notes	Investigation notes	✅
Timeline	Activity timeline	✅
Timeline	Auto-updates on commits	✅
Commands	`/commit finding`	✅
Commands	`/commit note`	✅
Commands	`/commit asset`	✅
Commands	`/commit evidence`	❌ v0.1
Reporting	Reporter summary generation	✅
Reporting	Markdown export	✅

Area	Feature	In v0.0
Reporting	PDF export	❌ v0.1
Reporting	Report versioning	❌ v0.1
Dashboard	Investigation cards	✅
Dashboard	Detailed metrics	❌ v0.1
Artifacts	Artifact storage	❌ v0.1
Visualization	Knowledge graph	❌ post-v1
Collaboration	Multi-user support	❌ post-v1
Database	SQLite	✅
Frontend	React + Zustand + xterm.js	✅
Backend	Python + FastAPI	✅

Stack

Layer	Technology
Frontend	React, xterm.js, Zustand
Backend	Python, FastAPI
Terminal	node-pty over WebSocket
Database	SQLite
AI	Anthropic API — two agents, streamed responses
Export	Markdown

v0.0 Success Criteria

Argus v0.0 is complete when a researcher can:

1. Create an investigation and get a unique code
2. Run real commands in a live terminal
3. Commit findings, notes, and assets via `/commit`
4. Have both agents respond with full awareness of the investigation state
5. Resume an investigation later with full context restored
6. Generate a Markdown report from structured findings

Everything else is v0.1 and beyond.

No auth (single researcher, local-style)

Argus v0.0 is a local-first offensive security workstation, not a hosted multi-user SaaS.

Authentication, JWTs, OAuth providers, user accounts, password resets, and multi-tenancy are explicitly out of scope.

Argus runs locally via:

argus start

and serves the UI on localhost.

All investigations, findings, notes, assets, reports, and settings are stored locally on the researcher's machine.

The operating system provides the primary security boundary.

Future versions may introduce optional authentication for shared/team deployments, but v0.0 should assume a single trusted user operating their own local workspace.

No login screen should exist in v0.0.

---

additional idea:

I wanna make sure that the AIs can understand the scope of bounty submission so when the researcher commits a finding through the terminal.

that means, import option exists so researcher imports scopes or any additional files related to the target domain focused in the investigation. Based on what is uploaded or existing subdomains or assets or files that are found through research, is added as a profile card (t-intel) of that target domain so it is easier for reviewing.

Target Intel

General

———————————

Target
Program
Platform
Status

Scope

———————————

In Scope

Out of Scope
Rules
Rewards

Infrastructure

Domains
Subdomains
IPs
URLs
Ports
Services

Technology

Frameworks
WAF
CDN
CMS
Languages
Cloud Provider

Recon

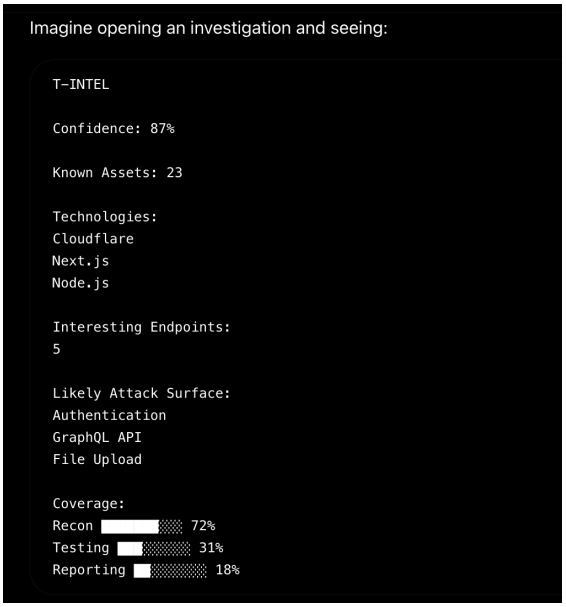
Interesting Endpoints
Parameters
Headers
Fingerprinting
Observations

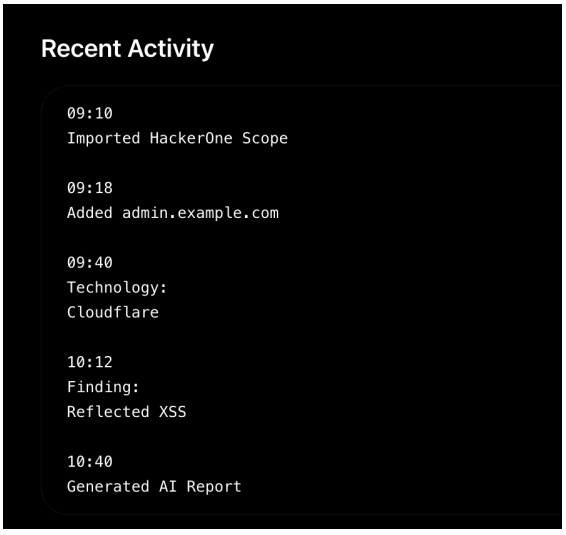
Findings Summary

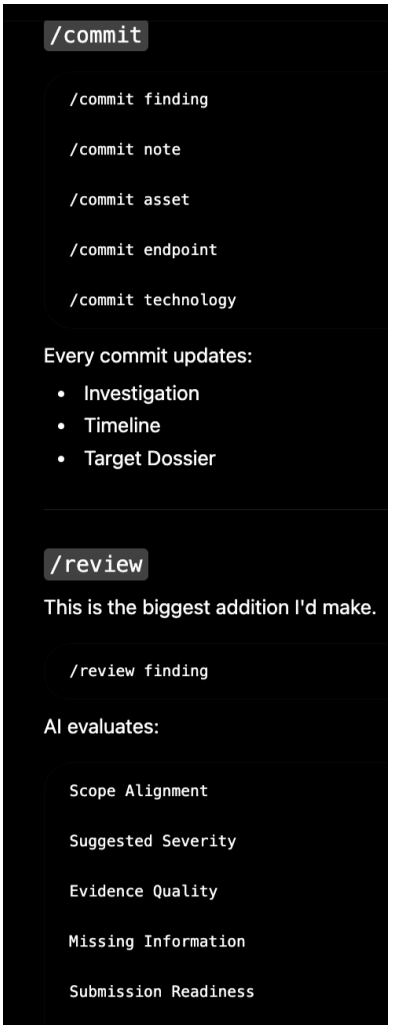
Open
Submitted
Resolved

Timeline Summary

Recent Activity







(there's a /review and /commit)

they can ask the AI (either one) about an analysis over their finding and whether it's under the scope and the evaluates whether the finding is severe/high/medium/low/informative and gives a reasoning which is based on the evidence in logs. the researcher can also export the terminal logs and AI written reports on what they found and is saved as a reports section of that investigation file so its easy access and can be maintained. we dont just

provide the AI written reports. The researcher himself can also create his own reports manually.